Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Hai, 5 tháng 10 2026, 2:52 PM
Kết thúc lúc	Thứ Hai, 5 tháng 10 2026, 2:55 PM
Thời gian thực hiện	2 phút 50 giây
Điểm	0,00/1,00
Điểm	**0,00** trên 10,00 (**0**%)

Câu hỏi **1**

Không trả lời

Đạt điểm 0,00 trên 1,00

Một máy chủ cần quản lý và thực thi các tác vụ (task) do người dùng gửi đến. Mỗi tác vụ có:

- một **mã định danh (ID)** duy nhất;
- một **độ ưu tiên (priority)**;
- một **hành động (action)** cần được thực thi.

Hãy hiện thực lớp `TaskScheduler` để quản lý và thực thi các tác vụ này.

Yêu cầu chức năng

Lớp `TaskScheduler` phải cung cấp các phương thức public sau:

```
void submit(String id, int priority, Runnable action)

void executeNext()

boolean cancel(String id)

int size()
```

1. Gửi tác vụ – `submit`

```
submit(String id, int priority, Runnable action): Thêm một tác vụ mới vào bộ lập lịch.
```

Một tác vụ hợp lệ phải thỏa mãn:

- `id` không được là `null` hoặc chuỗi rỗng;
- `priority` phải nằm trong khoảng từ **1 đến 10**, bao gồm cả hai đầu;
- `action` không được là `null`;
- không được tồn tại hai tác vụ **đang chờ thực thi** có cùng `id`.

Nếu tác vụ không hợp lệ, tác vụ phải bị từ chối và caller phải được thông báo về lỗi.

Nếu thao tác `submit()` thất bại, trạng thái của bộ lập lịch phải được giữ nguyên.

2. Thực thi tác vụ – `executeNext`

```
executeNext(): Thực thi và loại bỏ tác vụ tiếp theo khỏi bộ lập lịch.
```

Quy tắc lựa chọn tác vụ:

- tác vụ có `priority` cao hơn phải được thực thi trước;
- nếu hai tác vụ có cùng `priority`, tác vụ được gửi vào trước phải được thực thi trước.

Ví dụ, nếu các tác vụ được gửi theo thứ tự:

```
A, priority = 5
B, priority = 10
C, priority = 10
D, priority = 1
```

thì thứ tự thực thi phải là:

```
B → C → A → D
```

Nếu không còn tác vụ nào để thực thi, phương thức phải thông báo tình trạng này cho caller.

Caller phải có khả năng phân biệt giữa:

- không có tác vụ nào để thực thi; và
- có tác vụ, nhưng xảy ra lỗi trong quá trình thực thi `action` của tác vụ đó.

Nếu `action` của một tác vụ phát sinh lỗi, tác vụ đó **vẫn được xem là đã được thực thi** và không được còn tồn tại trong danh sách chờ.

Lỗi phát sinh trong quá trình thực thi tác vụ không được âm thầm bỏ qua.

3. Hủy tác vụ – `cancel`

```
cancel(String id): Hủy một tác vụ đang chờ dựa trên id.
```

Phương thức trả về:

```
true     nếu tìm thấy và hủy được tác vụ
false    nếu không có tác vụ đang chờ với ID tương ứng
```

Việc yêu cầu hủy một tác vụ không tồn tại **không được làm bộ lập lịch thất bại**.

Sau khi một tác vụ đã bị hủy, `id` của tác vụ đó có thể được sử dụng lại cho một tác vụ mới.

4. Số lượng tác vụ – `size`

```
size(): Trả về số lượng tác vụ hiện đang chờ thực thi.
```

Các tác vụ đã:

- thực thi thành công;
- thực thi nhưng phát sinh lỗi; hoặc
- bị hủy

không được tính vào kết quả của `size()`.

Yêu cầu thiết kế và hiện thực

Bộ lập lịch **không được cho phép code bên ngoài truy cập hoặc thay đổi trực tiếp cấu trúc dữ liệu nội bộ** dùng để lưu trữ các tác vụ.

Thiết kế cần đảm bảo rằng giới hạn độ ưu tiên tối đa hiện tại là `10` có thể được thay đổi trong tương lai mà không yêu cầu client code phải thay đổi.

Có thể sử dụng các class và interface thuộc **Java Standard Library**.

Ngoài API bắt buộc của `TaskScheduler`, sinh viên được phép tự định nghĩa thêm các:

- class;
- interface;
- exception;
- field;
- constructor;
- method

mà mình cho là cần thiết.

Không có yêu cầu về số lượng class hoặc cấu trúc dữ liệu phải sử dụng.

Quy định về Exception và thông báo lỗi

Khi xảy ra lỗi, các phương thức phải phát sinh đúng loại exception với nội dung thông báo (message) tương ứng như sau:

- Nếu `id == null` hoặc `id.isEmpty()` khi gọi `submit()`:
  - Exception: `IllegalArgumentException`
  - Message: `Task ID cannot be null or empty`
- Nếu `priority` nằm ngoài khoảng từ 1 đến 10:
  - Exception: `IllegalArgumentException`
  - Message: `Priority must be between 1 and 10`
- Nếu `action == null`:
  - Exception: `IllegalArgumentException`
  - Message: `Action cannot be null`
- Nếu đã tồn tại một tác vụ đang chờ có cùng `id`:
  - Exception: `IllegalArgumentException`
  - Message: `A waiting task with this ID already exists`
- Nếu gọi `executeNext()` khi không có tác vụ nào đang chờ:
  - Exception: `NoTaskAvailableException`
  - Message: `No task is available for execution`

Lưu ý:

- Message phải khớp chính xác với nội dung được quy định ở trên, bao gồm chữ hoa, chữ thường và khoảng trắng.
- Nếu một lời gọi `submit()` đồng thời vi phạm nhiều điều kiện, các điều kiện phải được kiểm tra theo thứ tự: `id` → `priority` → `action` → trùng `id`. Chỉ phát sinh exception tương ứng với điều kiện không hợp lệ đầu tiên.
- `NoTaskAvailableException` phải kế thừa `RuntimeException`.
- Nếu `action.run()` phát sinh `RuntimeException`, exception đó phải được truyền nguyên vẹn cho caller. Không được thay thế bằng exception khác, thay đổi message hoặc âm thầm bỏ qua exception.
- Nếu `id` truyền vào `cancel()` là `null`, chuỗi rỗng hoặc không tồn tại trong danh sách chờ, `cancel()` phải trả về `false` và không phát sinh exception.

- Không được tự in thông báo lỗi ra màn hình. Việc thông báo lỗi phải được thực hiện thông qua exception theo các quy định trên.

For example:

Test	Result

`System.out.println("=== TEST 1: PRIORITY ===");`
 	
`TaskScheduler scheduler = new TaskScheduler();`
 	
`scheduler.submit("backup", 3, () -> System.out.println("backup"));`
 	
`scheduler.submit("payment", 10, () -> System.out.println("payment"));`
 	
`scheduler.submit("email", 5, () -> System.out.println("email"));`
 	
`while (scheduler.size() > 0) {`
`  scheduler.executeNext();`
`}`
 	
`System.out.println();`
	`=== TEST 1: PRIORITY ===`
	`payment`
	`email`
	`backup`

`System.out.println("=== TEST 2: FIFO ===");`
 	
`TaskScheduler scheduler = new TaskScheduler();`
 	
`scheduler.submit("A", 5, () -> System.out.print("A "));`
 	
`scheduler.submit("B", 5, () -> System.out.print("B "));`
 	
`scheduler.submit("C", 5, () -> System.out.print("C "));`
 	
`while (scheduler.size() > 0) {`
`  scheduler.executeNext();`
`}`
 	
`System.out.println("\n");`
	`=== TEST 2: FIFO ===`
	`A B C`

`System.out.println("=== TEST 3: CANCEL ===");`
 	
`TaskScheduler scheduler = new TaskScheduler();`
 	
`scheduler.submit("A", 5, () -> System.out.println("A"));`
 	
`scheduler.submit("B", 10, () -> System.out.println("B"));`
 	
`System.out.println("Cancel B: " + scheduler.cancel("B"));`
 	
`System.out.println("Cancel X: " + scheduler.cancel("X"));`
 	
`scheduler.executeNext();`
 	
`System.out.println();`
	`=== TEST 3: CANCEL ===`
	`Cancel B: true`
	`Cancel X: false`
	`A`

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

```
public class TaskScheduler {
    // TODO
}
```